

Microservice Drift Tracker (MDT)

Project Title

Microservice Drift Tracker (MDT): An AI-Assisted Cross-Service Impact Analysis Framework for Microservice Architectures

Project Vision

Develop an intelligent software engineering platform that automatically detects how code changes in one microservice may affect other dependent microservices **before deployment**, helping developers identify risks early and reduce production failures.

Problem Statement

Modern applications consist of many interconnected microservices. Existing code review and static analysis tools primarily evaluate changes within a single repository or service and often lack awareness of dependencies across multiple services.

As a result, changes that appear safe locally can unintentionally introduce failures in downstream services, increasing deployment risk and debugging effort.

Proposed Solution

Microservice Drift Tracker combines:

- Git change analysis
- Dependency graph traversal
- Semantic code retrieval
- AI-assisted reasoning
- Deterministic risk scoring

to evaluate the potential impact of code changes across multiple microservices before deployment.

Core Objective

Automatically detect and explain the impact of source code changes across interconnected microservices using a hybrid deterministic and AI-assisted analysis pipeline.

Project Goals

- Detect code changes automatically
- Identify impacted services
- Predict deployment risks
- Explain why changes are risky
- Suggest corrective actions
- Visualize dependencies
- Improve software reliability

Target Users

- Software Developers
- Backend Engineers
- DevOps Engineers
- Software Architects
- QA Engineers
- Development Teams
- Research Community

Key Features

- GitHub webhook integration
- Git diff analysis
- AST-based code parsing
- Neo4j dependency graph
- ChromaDB semantic retrieval
- Hybrid Microservice Drift Algorithm (HMDA)
- LLM-generated explanations
- Explainable risk scoring

- Interactive React dashboard
 - End-to-end testing
-

Innovation (Novelty)

The novelty is **not** in using Git, Neo4j, vector databases, or LLMs individually. The contribution lies in **integrating these technologies into a unified workflow for cross-service impact prediction**.

Key innovations include:

- Cross-service impact analysis instead of single-repository analysis.
 - Hybrid deterministic + AI reasoning.
 - Retrieval-Augmented Generation (RAG) for contextual analysis.
 - Graph-based dependency propagation.
 - Explainable software engineering recommendations.
-

Unique Selling Proposition (USP)

Unlike conventional code review tools that analyze only changed files or repositories, MDT evaluates **how a change propagates through an entire microservice ecosystem** and provides:

- A deterministic impact score
 - Affected services
 - AI-generated explanations
 - Suggested fixes before deployment
-

Research Contribution

The project contributes a framework that combines:

- Software dependency analysis
- Graph databases
- Semantic retrieval
- Explainable AI

- Deterministic risk assessment

into a practical prototype for microservice impact prediction.

Patent Opportunity

Potential patentable contributions include:

- A novel dependency propagation algorithm for microservice impact prediction.
- A hybrid deterministic and AI-assisted risk scoring methodology.
- Architecture drift detection techniques.
- Explainable cross-service impact analysis workflows.

The patent focus should be on the **methodology or algorithm**, not on existing technologies like Neo4j or LLMs.

Research Gap

Current approaches often:

- Focus on a single repository.
- Ignore downstream service dependencies.
- Lack semantic understanding of code changes.
- Do not provide explainable cross-service risk analysis.

MDT aims to address these gaps by combining dependency analysis, semantic retrieval, and AI-assisted reasoning.

Core Technologies

Backend

- FastAPI
- Python

Frontend

- React
- TypeScript
- Tailwind CSS

- React Flow

Databases

- Neo4j
- ChromaDB

AI

- Large Language Model (e.g., GPT or Gemini)
- Embedding model

Infrastructure

- Docker
- GitHub Webhooks

System Architecture

Developer Push

|



GitHub Webhook

|



FastAPI Backend

|



Git Diff Analyzer

|

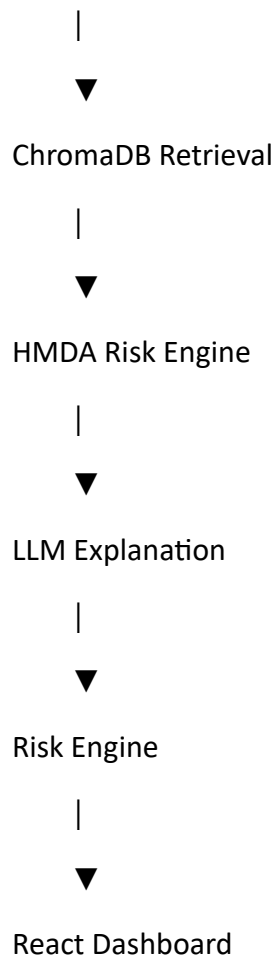


AST Parser

|



Neo4j Dependency Graph



Core Algorithm

HMDA — Hybrid Microservice Drift Algorithm

Inputs:

- Git diff
- AST metadata
- Dependency graph
- Retrieved contextual code
- Service metadata

Outputs:

- Risk score (0–100)
- Severity level
- Impacted services

- Confidence score
 - Explanation
 - Suggested fixes
-

Expected Outputs

- Risk classification (Low, Medium, High, Critical)
 - Dependency visualization
 - Affected service list
 - AI explanation
 - Suggested remediation
 - Historical analysis report
-

Research Methodology

1. Literature review
 2. Architecture design
 3. Prototype implementation
 4. Experimental evaluation
 5. Performance analysis
 6. Result validation
 7. Documentation and future work
-

Success Criteria

The project is successful if it can:

- Detect code changes automatically.
- Identify downstream service impacts.
- Produce deterministic risk scores.
- Generate contextual AI explanations.
- Complete analysis within an acceptable time for the prototype.

- Demonstrate the workflow through an interactive dashboard.

Business Value

For organizations, MDT can:

- Reduce deployment failures.
- Improve developer productivity.
- Increase confidence in code changes.
- Enhance software quality.
- Support CI/CD pipelines with intelligent pre-deployment analysis.

Future Scope

- Multi-language support
- Historical learning from commits
- CI/CD pipeline integration
- IDE plugins
- Kubernetes deployment
- Enterprise-scale repository support
- Automated architecture drift detection

One-Line Elevator Pitch

Microservice Drift Tracker is an AI-assisted software engineering platform that predicts and explains how code changes in one microservice may impact dependent services before deployment by combining dependency graphs, semantic code retrieval, and a deterministic risk assessment algorithm.

Project Highlights at a Glance

Aspect	Summary
Project Type	AI-Assisted Software Engineering / DevOps

Aspect	Summary
Domain	Microservices, Static Analysis, Explainable AI
Core Innovation	Cross-service impact prediction using graph analysis + RAG + deterministic risk scoring
Research Contribution	Hybrid Microservice Drift Algorithm (HMDA) framework
USP	Predicts downstream service impact, not just file-level code issues
Patent Potential	Novel dependency propagation and hybrid risk assessment methodology
Primary Technologies	FastAPI, React, Neo4j, ChromaDB, LLM, Docker
Target Users	Developers, DevOps, QA, Software Architects
Final Deliverable	Production-oriented prototype with dashboard, APIs, testing, and documentation
Long-Term Vision	An intelligent pre-deployment impact analysis platform for modern microservice ecosystems

MASTER PROJECT SPECIFICATION

Project Name

Microservice Drift Tracker (MDT): AI-Powered Cross-Service Impact Analysis System

Project Overview

The objective of this project is to build an intelligent software engineering platform that automatically detects whether code changes in one microservice can negatively impact dependent microservices before deployment.

The system must analyze source code changes, service dependencies, and semantic code context to generate an explainable risk assessment.

The project is intended as a research-grade prototype that combines graph databases, retrieval-augmented generation (RAG), deterministic risk scoring, and LLM-assisted reasoning.

The system is **not** a code review tool.

It is a **cross-service impact prediction system**.

Primary Goal

Given a Git commit,

the system should automatically determine

- Which microservice changed
- Which APIs changed
- Which dependent services may be affected
- Whether the change is backward compatible
- Probability of breaking other services
- Risk score
- Explanation
- Suggested fixes

before deployment.

Project Objectives

The system shall

- Detect GitHub pushes
 - Parse git diffs
 - Build and query dependency graphs
 - Retrieve relevant code using embeddings
 - Analyze semantic impact
 - Generate deterministic risk score
 - Generate AI explanation
 - Visualize affected services
-

System Workflow

Developer



Git Push



GitHub Webhook



FastAPI Backend



Git Diff Analyzer



AST Parser



Neo4j Dependency Graph



Retrieve Relevant Code from ChromaDB



Hybrid Microservice Drift Algorithm



LLM Reasoning



Risk Engine



React Dashboard

Prototype Scope

The prototype consists of four independent microservices.

User Service

Order Service

Payment Service

Notification Service

Each service must

- have its own repository
 - Dockerfile
 - REST APIs
 - own database
 - OpenAPI documentation
-

Tech Stack

Frontend

- React
- TypeScript
- Tailwind CSS
- React Flow

Backend

- FastAPI
- Python

AI

- GPT/Gemini
- LangChain
- Embedding Model

Database

- Neo4j
- ChromaDB

Infrastructure

- Docker
- Docker Compose
- GitHub Webhooks

Utilities

- GitPython

- Tree-sitter
-

Core Modules

GitHub Webhook

Receive push events.

Identify changed repository.

Trigger analysis.

Git Diff Analyzer

Extract

- changed files
 - changed methods
 - changed APIs
 - deleted endpoints
 - renamed functions
-

AST Parser

Parse source code.

Extract

- classes
- methods
- imports
- API endpoints

Avoid plain text parsing whenever possible.

Neo4j Dependency Engine

Maintain service dependency graph.

Example

User



Order



Payment



Notification

Return all affected downstream services.

ChromaDB Retrieval Engine

Store embeddings of

- methods
- controllers
- services
- interfaces

Retrieve only relevant code.

Never send the entire repository to the LLM.

Hybrid Microservice Drift Algorithm (HMDA)

This is the core contribution.

Input

- Git Diff

- Dependency Graph
- Retrieved Code
- Semantic Similarity
- API Metadata
- Service Criticality

Output

Impact Score

The algorithm—not the LLM—must own the final decision.

LLM

LLM responsibilities

- explain impact
- identify compatibility issues
- provide reasoning
- suggest fixes

LLM must never calculate the final risk score.

Risk Engine

Combine

- HMDA
- LLM confidence

Generate

Risk

LOW

MEDIUM

HIGH

CRITICAL

along with

0–100 score

Dashboard

Display

- Dependency graph
 - Changed service
 - Changed files
 - Affected services
 - Risk score
 - Confidence
 - AI explanation
 - Suggested fixes
 - Commit history
-

Folder Structure

backend/

frontend/

services/

neo4j/

chromadb/

algorithm/

embeddings/

docs/

docker/

tests/

scripts/

Development Principles

The project must follow

- Clean Architecture
- SOLID
- DRY
- KISS
- Modular Design

Every module must have a single responsibility.

Coding Standards

- Fully typed code
- Proper logging
- Proper exception handling
- Configuration through environment variables
- No duplicated logic

- Modular services
 - Comprehensive documentation
-

Testing

Every module must include

- Unit Tests
- Integration Tests
- End-to-End Tests

No feature is complete without tests.

Security

- Validate GitHub webhook signatures.
 - Never expose API keys.
 - Never hardcode secrets.
 - Use environment variables.
 - Validate all user inputs.
-

AI Rules

The AI must never

- invent APIs
- invent libraries
- invent database schemas
- invent endpoints
- invent configuration values

If information is missing,
ask for clarification instead of guessing.

Anti-Hallucination Rules

Always

- follow project architecture
- follow folder structure
- preserve existing interfaces
- explain assumptions
- validate dependencies
- keep deterministic logic outside the LLM

Never create files that are not required by the architecture.

Never modify unrelated modules.

Architecture Rules

Business logic

↓

Algorithm

↓

LLM

NOT

LLM

↓

Business Logic

The LLM is an assistant.

The algorithm owns decision making.

Definition of Done

Every completed feature must include

- implementation
- tests
- documentation

- logging
 - error handling
 - Docker compatibility
-

Future Scope

- Kubernetes
 - CI/CD
 - Automatic dependency discovery
 - Architecture drift detection
 - Historical learning
 - IDE plugins
 - Multi-language support
-

Final Instruction to Agentic AI

Treat this project as a **production-grade software engineering system**, not as a demo or CRUD application.

Prioritize:

1. Correctness over speed.
2. Deterministic logic over AI guesses.
3. Modularity over shortcuts.
4. Explainability over opaque decisions.
5. Extensibility so additional microservices, languages, and deployment environments can be added with minimal changes.

If requirements are ambiguous, **pause implementation of the affected part and request clarification instead of making assumptions**. This specification is the authoritative source for project structure and behavior.

For an Agentic AI, you don't just want to describe the project—you want to give it an **execution plan** so it knows *what to build, in what order, and what success looks like*. Here's a concise but comprehensive continuation you can append to the master brief.

PROJECT EXECUTION PLAN

Overall Development Strategy

The project must be built incrementally.

Each phase must be fully completed, tested, documented, and reviewed before moving to the next.

No future phase should begin until the current phase passes all acceptance criteria.

Phase 1 — Project Initialization

Goal

Set up the development environment and repository structure.

Tasks

- Create monorepo (or clearly documented multi-repo setup for the four services)
- Initialize Git
- Configure Docker Compose
- Configure environment variables
- Configure Python virtual environment
- Configure React frontend
- Configure FastAPI backend
- Configure linting and formatting
- Create README
- Create architecture documentation

Deliverables

- Working Docker Compose
 - Running React app
 - Running FastAPI server
 - Git repository initialized
 - Initial project documentation
-

Phase 2 — Build Demo Microservices

Create:

user-service

order-service

payment-service

notification-service

Each service must include

- REST API
- Swagger/OpenAPI
- Dockerfile
- Health endpoint
- Simple CRUD endpoints
- Unit tests

Define dependencies:

User



Order



Payment



Notification

Deliverable:

Four independent services communicating successfully.

Phase 3 — GitHub Integration

Build

GitHub Webhook Receiver

Tasks

- Receive push event
- Verify signature
- Parse payload
- Detect repository
- Detect branch
- Detect commit

Deliverable

Backend automatically receives GitHub push events.

Phase 4 — Git Diff Analyzer

Tasks

Extract

- changed files
- changed methods
- renamed methods
- deleted methods
- modified APIs

Avoid relying on simple line-based parsing where structured parsing is available.

Deliverable

Structured JSON representing code changes.

Example

```
{  
  "service": "payment",  
  "changedFunctions": [  
    "processPayment"  
  ],  
  "changedEndpoints": [  
    "/payment/process"  
  ]  
}
```

Phase 5 — AST Parser

Implement Tree-sitter.

Extract

- functions
- classes
- interfaces
- annotations
- API routes
- imports

Output standardized metadata.

Deliverable

Language-aware code parsing.

Phase 6 — Neo4j Integration

Create graph

(Service)



(API)



Method

Relationships

CALLS

DEPENDS_ON

USES

EXPOSES

Deliverable

Dependency graph visualization.

Phase 7 — Embedding Pipeline

Create embeddings for

- methods
- classes
- APIs
- controllers

Store inside ChromaDB.

Deliverable

Searchable vector database.

Phase 8 — Retrieval Engine

Input

Changed Method

Retrieve

Top relevant

- APIs
- classes
- methods
- services

Deliverable

Relevant context only.

Phase 9 — HMDA Algorithm

Implement

Hybrid Microservice Drift Algorithm.

Input

- Git Diff
- Neo4j Graph
- Retrieved Context
- Metadata

Output

Impact Score

0–100

Algorithm responsibilities

- dependency weighting
- semantic drift

- API criticality
- service criticality
- historical weight (future)

The algorithm must remain deterministic.

Phase 10 — LLM Integration

Provide

- changed APIs
- retrieved code
- dependency information

LLM returns

- explanation
- compatibility analysis
- suggested fixes

Never allow the LLM to directly determine the final risk score.

Phase 11 — Risk Engine

Combine

HMDA

-

LLM confidence

Generate

Risk

LOW

MEDIUM

HIGH

CRITICAL

Also generate

- confidence
- explanation
- affected services

Phase 12 — Dashboard

Display

- Dependency graph
- Commit
- Changed files
- Risk score
- AI explanation
- Suggested fixes
- Timeline
- History

Deliverable

Production-style dashboard.

Phase 13 — Testing

Implement

Unit Tests

Integration Tests

API Tests

End-to-End Tests

Regression Tests

Every feature must be tested.

Phase 14 — Documentation

Generate

- API docs
- Developer guide
- User guide
- Architecture diagrams
- Deployment guide

Documentation is mandatory.

PROJECT RULES

Always follow these rules.

Rule 1

Never skip tests.

Rule 2

Never hardcode configuration.

Rule 3

Never duplicate business logic.

Rule 4

Use modular architecture.

Rule 5

Keep AI reasoning independent from deterministic logic.

Rule 6

Every module must have a single responsibility.

Rule 7

Every public function must be documented.

Rule 8

Every API must return proper status codes.

Rule 9

Every exception must be logged.

Rule 10

Every feature must include documentation.

Agent Responsibilities

The AI agent should

- create clean folder structures
 - create production-quality code
 - create reusable modules
 - write tests
 - generate documentation
 - keep architecture consistent
-

Things the Agent MUST NEVER DO

Never

- invent APIs
- invent endpoints

- invent schemas
 - invent services
 - invent libraries
 - ignore architecture
 - skip tests
 - skip logging
 - mix frontend and backend concerns
 - bypass deterministic validation
-

Success Criteria

The project is complete only if:

- Four microservices communicate successfully.
 - GitHub pushes automatically trigger analysis.
 - Git diffs are parsed correctly.
 - Neo4j accurately identifies dependent services.
 - ChromaDB retrieves only relevant code.
 - HMDA computes a deterministic impact score.
 - The LLM provides explanations and suggested fixes.
 - The Risk Engine produces a final severity classification.
 - The React dashboard displays all analysis results.
 - Docker Compose launches the full stack with a single command.
 - End-to-end tests demonstrate that a change in one service (for example, a breaking API change in payment-service) is detected and reported as affecting downstream services before deployment.
-

Final Instruction to the Agent

Treat this as a **production-oriented engineering prototype** with research value. Build each phase in order, keep modules loosely coupled and well documented, and do not proceed to the next phase until the current one is implemented, tested, and verified against its

acceptance criteria. The architecture, deterministic HMDA algorithm, and explainable analysis are the core priorities; the LLM is a supporting component, not the primary decision maker.

These sections are exactly what a software architect would include in a Software Design Document (SDD). You can paste the following directly into your project documentation.

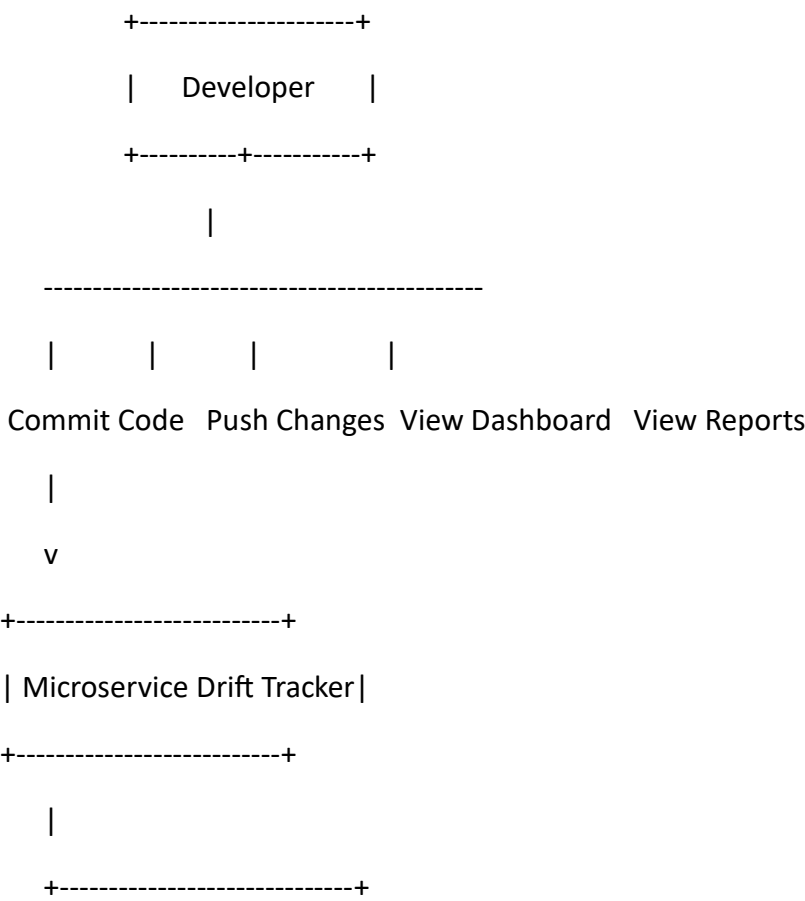
1. UML DIAGRAMS

1.1 Use Case Diagram

Actors

- Developer
- GitHub
- System Administrator
- AI Engine (LLM)
- Project Dashboard User

Use Cases



```
| Analyze Git Changes      |
| Build Dependency Graph   |
| Retrieve Relevant Code    |
| Calculate Risk           |
| Generate AI Explanation  |
| Store Analysis Report    |
+-----+
```

Administrator

```
|
+----- Configure Services
|
+----- Manage Dependencies
|
+----- Monitor Logs
```

1.2 Class Diagram

```
+-----+
| RepositoryManager |
+-----+
| cloneRepo()      |
| pullChanges()    |
| getCommit()      |
+-----+

|

|
```

+-----+

| GitDiffAnalyzer |

+-----+

| parseDiff() |

| detectChanges() |

+-----+

|

|

+-----+

| DependencyEngine |

+-----+

| buildGraph() |

| traverseGraph() |

+-----+

|

|

+-----+

| RetrievalEngine |

+-----+

| embedCode() |

| retrieveContext() |

+-----+

|

|

+-----+

| HMDAEngine |

+-----+

| calculateRisk() |

+-----+

|

|

+-----+

| LLMService |

+-----+

| analyzeImpact() |

| suggestFix() |

+-----+

|

|

+-----+

| ReportGenerator |

+-----+

| generateReport() |

+-----+

1.3 Component Diagram

React Dashboard

↓

FastAPI Backend

└─ Git Module

└─ Neo4j Module

└─ ChromaDB Module

└─ HMDA Engine

└─ LLM Module

└─ Report Module

↓

Docker Compose



Microservices

1.4 Deployment Diagram

Developer



GitHub



Webhook Server



FastAPI



Neo4j



ChromaDB



LLM API



React Dashboard

2. SEQUENCE DIAGRAM

Developer



| Push Code



GitHub



| Webhook



FastAPI Backend



| Parse Git Diff

v

Git Diff Analyzer

|

| Changed APIs

v

Neo4j

|

| Find Dependencies

v

ChromaDB

|

| Retrieve Context

v

HMDA Algorithm

|

| Risk Score

v

LLM

|

| Explanation

v

Risk Engine

|

| Final Report

v

Dashboard

Sequence Description

1. Developer pushes code.

2. GitHub sends webhook.
3. Backend receives payload.
4. Git Diff Analyzer extracts changes.
5. Dependency graph identifies impacted services.
6. ChromaDB retrieves relevant code.
7. HMDA computes deterministic score.
8. LLM generates explanation.
9. Risk engine combines results.
10. Dashboard displays findings.

3. DATABASE SCHEMA

Neo4j

Nodes

Service

Service

id

name

repository

version

owner

API

API

id

path

method

serviceId

Class

Class

id

name

package

Method

Method

id

name

signature

Relationships

(Service)-[:DEPENDS_ON]->(Service)

(Service)-[:EXPOSES]->(API)

(API)-[:CALLS]->(API)

(Class)-[:CONTAINS]->(Method)

(Method)-[:CALLS]->(Method)

(Service)-[:USES]->(Class)

ChromaDB Collections

code_chunks

id

service

file

class

method

language

embedding

content

api_embeddings

apild

service

endpoint

embedding

description

documentation

documentId

title

embedding

content

Analysis Reports (SQL)

AnalysisReport

id

commitId

service

riskScore

severity

confidence

timestamp

4. API CONTRACTS

POST /webhook/github

Purpose

Receive GitHub webhook.

Request

```
{  
  "repository": "payment-service",  
  "commit": "abc123"  
}
```

Response

```
{  
  "status": "accepted"  
}
```

POST /analyze

Request

```
{  
  "repository": "payment-service",  
  "commit": "abc123"  
}
```

Response

```
{  
  "analysisId":"1001"  
}
```

GET /analysis/{id}

Response

```
{  
  "riskScore":82,  
  "severity":"HIGH",  
  "affectedServices":[  
    "order-service",  
    "notification-service"  
  ],  
  "explanation":"Payment API changed..."  
}
```

GET /dependencies/{service}

Response

```
{  
  "service":"payment-service",  
  "dependsOn":[  
    "notification-service"  
  ],  
  "usedBy":[  
    "order-service"  
  ]  
}
```

POST /embed

Purpose

Generate embeddings.

Response

```
{  
  "status": "completed"  
}
```

GET /health

```
{  
  "status": "healthy"  
}
```

5. ROADMAP

Phase 1

Environment Setup

- Docker
 - FastAPI
 - React
 - Git
-

Phase 2

Microservices

- User
- Order
- Payment
- Notification

Phase 3

GitHub Integration

- Webhooks
- Git Diff

Phase 4

Neo4j Graph

- Nodes
- Relationships
- Queries

Phase 5

Embeddings

- Chunking
- ChromaDB
- Retrieval

Phase 6

HMDA

- Dependency Weight
- API Weight
- Semantic Drift
- Risk Score

Phase 7

LLM

- Prompt Engineering

- Explanation
 - Suggested Fixes
-

Phase 8

Dashboard

- Graph Visualization
 - Reports
 - Analytics
-

Phase 9

Testing

- Unit
 - Integration
 - E2E
-

Phase 10

Deployment

- Docker Compose
 - Cloud Deployment
-

6. TESTING STRATEGY

Unit Testing

Verify

- Git parser
- HMDA
- Graph traversal
- Retrieval

Tools

pytest

unittest

Integration Testing

Verify

GitHub

↓

Backend

↓

Neo4j

↓

LLM

works together.

End-to-End Testing

Scenario

Developer changes Payment API.

Expected

- Webhook triggers
- Diff extracted
- Dependencies identified

- Relevant context retrieved
 - HMDA computes score
 - LLM explains impact
 - Dashboard shows warning
-

Performance Testing

Measure

- Analysis latency
- Graph query time
- Retrieval time
- End-to-end execution time

Target

<10 seconds

for the prototype.

Security Testing

Validate

- Webhook signatures
 - API authentication
 - Secret management
 - Input validation
 - Injection resistance
-

Regression Testing

Ensure new features do not break:

- Dependency traversal
- Risk scoring
- Retrieval

- Dashboard rendering
-

7. RESEARCH METHODOLOGY

Research Type

Applied Software Engineering Research with an AI-assisted analysis component.

Research Problem

Current code review and static analysis tools primarily focus on individual repositories and often lack automated reasoning about the downstream impact of changes across interconnected microservices before deployment.

Research Objectives

1. Detect cross-service impact automatically.
 2. Improve developer awareness of risky changes.
 3. Reduce deployment failures caused by dependency changes.
 4. Provide explainable AI-assisted recommendations.
-

Methodology

Step 1 — Literature Review

Study:

- Static code analysis
- Dependency analysis
- Software architecture drift
- Retrieval-Augmented Generation (RAG)
- Graph databases in software engineering

Step 2 — Prototype Design

Design the architecture with:

- FastAPI
- Neo4j

- ChromaDB
- LLM
- HMDA

Step 3 — Implementation

Build the prototype incrementally according to the roadmap.

Step 4 — Experimental Evaluation

Prepare representative scenarios, for example:

- Non-breaking internal refactor
- Breaking API signature change
- Endpoint removal
- Added required request parameter
- Cascading dependency changes

For each scenario, measure:

- Impact detection accuracy
- Precision and recall of affected-service identification
- False positives and false negatives
- End-to-end analysis latency
- Developer usefulness (qualitative feedback)

Step 5 — Result Analysis

Compare predicted impacts with the known ground truth for each scenario. Analyze strengths, limitations, and cases where the system overestimates or underestimates risk.

Step 6 — Conclusions and Future Work

Summarize findings, discuss limitations of the prototype, and propose future enhancements such as automatic dependency discovery, historical learning, multi-language support, CI/CD integration, and IDE plugins.

These sections provide a solid foundation for a final-year project report and for directing an AI coding agent. As you implement the prototype, update them with concrete details from your codebase, APIs, schemas, and evaluation results rather than treating them as fixed documentation.

With a **team of 3**, the project plan should emphasize ownership and parallel work while keeping communication simple. Here's a revised management plan.

Project Management Plan (3-Person Team)

Team Structure

Member	Primary Role	Secondary Role	Responsibilities
Member 1	Project Lead & Backend Engineer	DevOps	Project planning, FastAPI, GitHub Webhooks, HMDA algorithm, Docker, deployment, code reviews
Member 2	AI & Database Engineer	Backend	Neo4j, ChromaDB, embeddings, retrieval pipeline, LLM integration, database schema
Member 3	Frontend & QA Engineer	Documentation	React dashboard, React Flow visualization, API integration, testing, documentation, demo preparation

Responsibility Matrix (RACI)

Module	Member 1	Member 2	Member 3
Project Planning	A/R	C	C
FastAPI Backend	R	C	I
GitHub Webhooks	R	I	I
Git Diff Parser	R	C	I
Tree-sitter Integration	R	C	I
Neo4j	C	R	I
ChromaDB	C	R	I
Embedding Pipeline	I	R	I
LLM Integration	C	R	I
HMDA Algorithm	R	C	I

Module	Member 1	Member 2	Member 3
Dashboard	I	C	R
API Integration	C	C	R
Testing	C	C	R
Docker Compose	R	C	I
Documentation	C	C	R
Final Presentation	R	R	R

Legend:

- **R** – Responsible
- **A** – Accountable
- **C** – Consulted
- **I** – Informed

Development Timeline (12 Weeks)

Phase 1 (Week 1)

Project Setup

Member 1

- Repository setup
- FastAPI initialization
- Docker Compose

Member 2

- Neo4j setup
- ChromaDB setup
- Database schema

Member 3

- React project setup
- Tailwind CSS

- Dashboard wireframes

Deliverable: Basic project skeleton running locally.

Phase 2 (Weeks 2–3)

Build Microservices

Member 1

- User Service
- Order Service

Member 2

- Payment Service
- Notification Service
- Neo4j dependency graph

Member 3

- API testing
- Dashboard layout
- Service cards

Deliverable: Four communicating microservices.

Phase 3 (Weeks 4–5)

Backend Intelligence

Member 1

- GitHub Webhooks
- Git Diff Analyzer
- Tree-sitter parsing

Member 2

- Embedding generation
- ChromaDB indexing
- Neo4j traversal

Member 3

- Dashboard integration
- Dependency visualization

Deliverable: Automated change detection and dependency analysis.

Phase 4 (Weeks 6–8)

AI Pipeline

Member 1

- HMDA implementation
- Risk calculation

Member 2

- LLM integration
- Prompt engineering
- Retrieval pipeline

Member 3

- Risk dashboard
- Reports
- History view

Deliverable: End-to-end impact analysis.

Phase 5 (Weeks 9–10)

System Integration

All members collaborate on:

- Integration testing
- Bug fixing
- Performance tuning
- UI improvements
- Logging

Deliverable: Fully integrated prototype.

Phase 6 (Weeks 11–12)

Finalization

Member 1

- Deployment
- Docker validation
- Architecture documentation

Member 2

- Research methodology
- Algorithm documentation
- Evaluation metrics

Member 3

- Final report
- Presentation
- User guide
- Demo recording

Deliverable: Final project ready for demonstration and submission.

Daily Workflow

Daily Stand-up (15 Minutes)

Each member answers:

1. What did I complete yesterday?
 2. What will I work on today?
 3. What blockers do I have?
-

Weekly Sprint Review

- Demonstrate completed features.

- Review code quality.
- Resolve blockers.
- Plan next week's work.

Git Workflow

main

|

├— develop

|

├— feature/backend

├— feature/ai

└— feature/frontend

↓

Pull Request

↓

Code Review

↓

Merge into develop

↓

Release to main

Milestones

Week Milestone

- 1 Project initialized
- 3 Four microservices operational
- 5 GitHub + dependency graph working

Week Milestone

- 8 AI analysis pipeline completed
- 10 Dashboard integrated
- 11 Testing completed
- 12 Final deployment and presentation

Risk Management

Risk	Owner	Mitigation
GitHub webhook issues	Member 1	Validate signatures, add retries
Neo4j modeling issues	Member 2	Start with a simple graph and expand incrementally
LLM inconsistency	Member 2	Keep HMDA as the final decision maker
Dashboard integration bugs	Member 3	Use mocked APIs before backend is complete
Schedule delays	Project Lead	Reprioritize backlog and focus on core MVP

Definition of Done

A task is complete only when:

- Code is implemented.
- Unit tests pass.
- Feature is integrated successfully.
- Documentation is updated.
- Code has been reviewed by at least one other team member.
- Docker Compose runs successfully with the new changes.

Final Delivery Checklist

By the end of the project, the team should deliver:

- Four Dockerized microservices
- FastAPI orchestration backend
- GitHub webhook integration
- Git diff and AST analysis
- Neo4j dependency graph
- ChromaDB retrieval system
- HMDA risk scoring engine
- LLM-powered explanations
- React dashboard with dependency visualization
- Test suite (unit, integration, end-to-end)
- Complete documentation
- Deployment guide
- Final presentation and demonstration

This division keeps each member focused on a primary domain while ensuring enough overlap for reviews and integration, making it practical for a **3-person capstone team**.

Risk Management & Deadlines

Project: Microservice Drift Tracker (MDT)

Team Size: 3 Members

Duration: 12 Weeks

1. Risk Management Plan

Risk Categories

The project risks are divided into:

- Technical Risks
- Project Management Risks
- Team Risks
- Research Risks
- Infrastructure Risks

- Deployment Risks

Each risk is evaluated using:

- **Probability (P):** Low / Medium / High
- **Impact (I):** Low / Medium / High
- **Priority:** $P \times I$
- **Mitigation Strategy**
- **Contingency Plan**
- **Owner**

Risk Register

ID	Risk	Probability	Impact	Priority	Owner	Mitigation	Contingency
R1	GitHub Webhook not triggering	Medium	High	High	Member 1	Validate signatures, add retry logic	Manual webhook trigger for testing
R2	Incorrect Git diff parsing	Medium	High	High	Member 1	Use GitPython with unit tests	Manual verification of diffs
R3	Tree-sitter parser compatibility issues	Medium	Medium	Medium	Member 1	Support one language first (e.g., Python or Java)	Fallback to basic parsing for prototype
R4	Neo4j dependency graph modeled incorrectly	Medium	High	High	Member 2	Validate graph with sample scenarios	Simplify graph relationships
R5	ChromaDB returns irrelevant code	High	Medium	High	Member 2	Fine-tune chunk size and retrieval	Use keyword search fallback

ID	Risk	Probability	Impact	Priority	Owner	Mitigation	Contingency
R6	LLM hallucinations	High	Medium	High	Member 2	Restrict prompts and retrieved context	Ignore unsupported AI claims; HMDA decides
R7	HMDA produces inaccurate scores	Medium	High	High	Member 1	Validate against predefined scenarios	Adjust weights and thresholds
R8	Frontend-backend API mismatch	Medium	Medium	Medium	Member 3	Define API contracts early	Mock APIs until backend stabilizes
R9	Docker configuration issues	Medium	Medium	Medium	Member 1	Use Docker Compose from the start	Run services individually
R10	Team member unavailable	Low	High	Medium	PM/Team	Cross-train members and document work	Redistribute tasks
R11	Sprint delays	Medium	High	High	PM/Team	Weekly reviews and backlog prioritization	Reduce scope to core MVP
R12	Integration failures	Medium	High	High	All	Continuous integration and early integration testing	Allocate dedicated integration week
R13	Performance slower than target	Medium	Medium	Medium	Member 2	Cache graph queries and optimize retrieval	Limit dataset size for prototype

ID	Risk	Probability	Impact	Priority	Owner	Mitigation	Contingency
R14	Research novelty questioned	Medium	High	High	PM	Clearly define HMDA contribution and evaluation	Emphasize workflow integration and explainability
R15	Documentation incomplete	Low	Medium	Low	Member 3	Update docs each sprint	Reserve final week for documentation

Top 5 Critical Risks

1. LLM Hallucination

Impact: AI may generate incorrect explanations.

Mitigation:

- Use Retrieval-Augmented Generation (RAG).
- Restrict prompts to retrieved context.
- HMDA remains the authoritative decision engine.

2. Dependency Graph Errors

Impact:

Incorrect downstream services may be identified.

Mitigation:

- Validate relationships manually during development.
- Use predefined dependency scenarios for testing.

3. Git Diff Parsing Errors

Impact:

Code changes may be interpreted incorrectly.

Mitigation:

- Test with multiple commit scenarios.

- Compare parsed output with GitHub diffs.

4. Project Schedule Slippage

Impact:

Incomplete MVP by the deadline.

Mitigation:

- Prioritize core functionality.
- Defer non-essential features (e.g., Kubernetes, IDE plugin).

5. Integration Issues

Impact:

Independent modules fail when combined.

Mitigation:

- Integrate continuously rather than waiting until the end.
- Maintain stable API contracts.

Risk Response Strategy

Strategy When to Use

Avoid Redesign to eliminate the risk (e.g., reduce supported languages)

Mitigate Reduce probability or impact (e.g., testing, reviews)

Transfer Use managed services or external tooling where appropriate

Accept Accept low-impact risks with monitoring

Project Deadlines (12-Week Schedule)

Week	Milestone	Deliverables
Week 1	Project Initiation	Repository, architecture, Docker, environment setup

Week	Milestone	Deliverables
Week 2	Microservice Development Begins	User and Order services
Week 3	Microservices Complete	Payment and Notification services, API documentation
Week 4	GitHub Integration	Webhooks, commit detection
Week 5	Git Analysis	Git diff parser, Tree-sitter integration
Week 6	Dependency Graph	Neo4j schema, graph traversal
Week 7	Retrieval Pipeline	ChromaDB indexing and semantic retrieval
Week 8	HMDA Implementation	Risk calculation engine
Week 9	AI Integration	LLM explanations and recommendations
Week 10	Dashboard	React UI, dependency visualization
Week 11	Testing & Integration	Unit, integration, and end-to-end testing
Week 12	Final Delivery	Documentation, deployment, demonstration, report

Internal Weekly Deliverables

Week 1

- Repository created
- Docker Compose operational
- Project structure finalized

Week 2

- User Service complete
- Order Service complete

Week 3

- Payment Service complete

- Notification Service complete
- Services communicate successfully

Week 4

- GitHub webhook operational

Week 5

- Git diff analyzer operational
- AST parser extracts structured metadata

Week 6

- Neo4j graph populated
- Dependency queries validated

Week 7

- ChromaDB populated
- Retrieval tested with representative queries

Week 8

- HMDA computes deterministic risk scores

Week 9

- LLM provides explanations using retrieved context

Week 10

- Dashboard displays dependency graph and risk analysis

Week 11

- All automated tests pass
- Performance targets evaluated

Week 12

- Final report completed
- Live demonstration prepared
- Deployment guide finalized

Acceptance Criteria by Milestone

Milestone	Acceptance Criteria
Project Setup	Development environment runs with one command
Microservices	All four services expose documented REST APIs
Git Integration	Push events trigger analysis automatically
Dependency Graph	Downstream services identified correctly
Retrieval	Relevant code is returned for changed components
HMDA	Deterministic risk score generated for each change
AI	Explanations reference retrieved context and avoid unsupported claims
Dashboard	Displays risk score, explanation, and affected services
Testing	Unit, integration, and end-to-end tests pass
Final Delivery	Documentation, code, and demonstration are complete

Contingency Plan

If the project falls behind schedule:

1. Priority 1 (Must Have)

- Four microservices
- GitHub Webhooks
- Git Diff Analyzer
- Neo4j dependency graph
- HMDA risk engine

2. Priority 2 (Should Have)

- ChromaDB retrieval
- LLM explanations
- Dashboard analytics

3. Priority 3 (Could Have)

- Historical learning

- Multi-language support
- Kubernetes deployment
- IDE plugin
- Advanced analytics

This prioritization ensures that the core research contribution—the deterministic cross-service impact analysis prototype—is completed even if time constraints require deferring optional enhancements.

Risk Management & Deadlines

Project: Microservice Drift Tracker (MDT)

Team Size: 3 Members

Duration: 12 Weeks

1. Risk Management Plan

Risk Categories

The project risks are divided into:

- Technical Risks
- Project Management Risks
- Team Risks
- Research Risks
- Infrastructure Risks
- Deployment Risks

Each risk is evaluated using:

- **Probability (P):** Low / Medium / High
 - **Impact (I):** Low / Medium / High
 - **Priority:** $P \times I$
 - **Mitigation Strategy**
 - **Contingency Plan**
 - **Owner**
-

Risk Register

ID	Risk	Probability	Impact	Priority	Owner	Mitigation	Contingency
R1	GitHub Webhook not triggering	Medium	High	High	Member 1	Validate signatures, add retry logic	Manual webhook trigger for testing
R2	Incorrect Git diff parsing	Medium	High	High	Member 1	Use GitPython with unit tests	Manual verification of diffs
R3	Tree-sitter parser compatibility issues	Medium	Medium	Medium	Member 1	Support one language first (e.g., Python or Java)	Fallback to basic parsing for prototype
R4	Neo4j dependency graph modeled incorrectly	Medium	High	High	Member 2	Validate graph with sample scenarios	Simplify graph relationships
R5	ChromaDB returns irrelevant code	High	Medium	High	Member 2	Fine-tune chunk size and retrieval	Use keyword search fallback
R6	LLM hallucinations	High	Medium	High	Member 2	Restrict prompts and retrieved context	Ignore unsupported AI claims; HMDA decides
R7	HMDA produces inaccurate scores	Medium	High	High	Member 1	Validate against predefined scenarios	Adjust weights and thresholds
R8	Frontend-backend API mismatch	Medium	Medium	Medium	Member 3	Define API contracts early	Mock APIs until backend stabilizes

ID	Risk	Probability	Impact	Priority	Owner	Mitigation	Contingency
R9	Docker configuration issues	Medium	Medium	Medium	Member 1	Use Docker Compose from the start	Run services individually
R10	Team member unavailable	Low	High	Medium	PM/Team	Cross-train members and document work	Redistribute tasks
R11	Sprint delays	Medium	High	High	PM/Team	Weekly reviews and backlog prioritization	Reduce scope to core MVP
R12	Integration failures	Medium	High	High	All	Continuous integration and early integration testing	Allocate dedicated integration week
R13	Performance slower than target	Medium	Medium	Medium	Member 2	Cache graph queries and optimize retrieval	Limit dataset size for prototype
R14	Research novelty questioned	Medium	High	High	PM	Clearly define HMDA contribution and evaluation	Emphasize workflow integration and explainability
R15	Documentation incomplete	Low	Medium	Low	Member 3	Update docs each sprint	Reserve final week for documentation

Top 5 Critical Risks

1. LLM Hallucination

Impact: AI may generate incorrect explanations.

Mitigation:

- Use Retrieval-Augmented Generation (RAG).
 - Restrict prompts to retrieved context.
 - HMDA remains the authoritative decision engine.
-

2. Dependency Graph Errors

Impact:

Incorrect downstream services may be identified.

Mitigation:

- Validate relationships manually during development.
 - Use predefined dependency scenarios for testing.
-

3. Git Diff Parsing Errors

Impact:

Code changes may be interpreted incorrectly.

Mitigation:

- Test with multiple commit scenarios.
 - Compare parsed output with GitHub diffs.
-

4. Project Schedule Slippage

Impact:

Incomplete MVP by the deadline.

Mitigation:

- Prioritize core functionality.
 - Defer non-essential features (e.g., Kubernetes, IDE plugin).
-

5. Integration Issues

Impact:

Independent modules fail when combined.

Mitigation:

- Integrate continuously rather than waiting until the end.
- Maintain stable API contracts.

Risk Response Strategy

Strategy When to Use

Avoid Redesign to eliminate the risk (e.g., reduce supported languages)

Mitigate Reduce probability or impact (e.g., testing, reviews)

Transfer Use managed services or external tooling where appropriate

Accept Accept low-impact risks with monitoring

Project Deadlines (12-Week Schedule)

Week	Milestone	Deliverables
Week 1	Project Initiation	Repository, architecture, Docker, environment setup
Week 2	Microservice Development Begins	User and Order services
Week 3	Microservices Complete	Payment and Notification services, API documentation
Week 4	GitHub Integration	Webhooks, commit detection
Week 5	Git Analysis	Git diff parser, Tree-sitter integration
Week 6	Dependency Graph	Neo4j schema, graph traversal
Week 7	Retrieval Pipeline	ChromaDB indexing and semantic retrieval
Week 8	HMDA Implementation	Risk calculation engine
Week 9	AI Integration	LLM explanations and recommendations

Week	Milestone	Deliverables
Week 10	Dashboard	React UI, dependency visualization
Week 11	Testing & Integration	Unit, integration, and end-to-end testing
Week 12	Final Delivery	Documentation, deployment, demonstration, report

Internal Weekly Deliverables

Week 1

- Repository created
- Docker Compose operational
- Project structure finalized

Week 2

- User Service complete
- Order Service complete

Week 3

- Payment Service complete
- Notification Service complete
- Services communicate successfully

Week 4

- GitHub webhook operational

Week 5

- Git diff analyzer operational
- AST parser extracts structured metadata

Week 6

- Neo4j graph populated
- Dependency queries validated

Week 7

- ChromaDB populated
- Retrieval tested with representative queries

Week 8

- HMDA computes deterministic risk scores

Week 9

- LLM provides explanations using retrieved context

Week 10

- Dashboard displays dependency graph and risk analysis

Week 11

- All automated tests pass
- Performance targets evaluated

Week 12

- Final report completed
- Live demonstration prepared
- Deployment guide finalized

Acceptance Criteria by Milestone

Milestone	Acceptance Criteria
Project Setup	Development environment runs with one command
Microservices	All four services expose documented REST APIs
Git Integration	Push events trigger analysis automatically
Dependency Graph	Downstream services identified correctly
Retrieval	Relevant code is returned for changed components
HMDA	Deterministic risk score generated for each change
AI	Explanations reference retrieved context and avoid unsupported claims

Milestone	Acceptance Criteria
Dashboard	Displays risk score, explanation, and affected services
Testing	Unit, integration, and end-to-end tests pass
Final Delivery	Documentation, code, and demonstration are complete

Contingency Plan

If the project falls behind schedule:

1. Priority 1 (Must Have)

- Four microservices
- GitHub Webhooks
- Git Diff Analyzer
- Neo4j dependency graph
- HMDA risk engine

2. Priority 2 (Should Have)

- ChromaDB retrieval
- LLM explanations
- Dashboard analytics

3. Priority 3 (Could Have)

- Historical learning
- Multi-language support
- Kubernetes deployment
- IDE plugin
- Advanced analytics

This prioritization ensures that the core research contribution—the deterministic cross-service impact analysis prototype—is completed even if time constraints require deferring optional enhancements.